

CHETTINAD VIDYASHRAM



COMPUTER SCIENCE PROJECT

STD XII (2024-25)

NAME:

SECTION:

TITLE:



Chettinad Vidyashram

(Affiliated to Central Board of Secondary Education, New Delhi)
(Chettinad House, R.A.Puram, Chennai – 600 028)

COMPUTER SCIENCE

Certified to be the Bonafide Record of work done by

_____ of Std XII Sec _____

in the Computer Science Lab of the CHETTINAD VIDYASHRAM,
CHENNAI, during the year 2024 – 2025.

Date:

Teacher-in-charge

REGISTER NO. _____

Submitted for All India Senior Secondary Practical Examination in

Computer Science held on _____ at

Chettinad Vidyashram, Chennai – 600 028.

Principal

Internal Examiner

External Examiner

PYTHON OVERVIEW

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python is popular in various domains, from web development to data science, due to its ease of learning, large community, and extensive libraries. Here's a brief overview:

Key Features of Python:

1. **Simple Syntax:** Python's syntax is clear and easy to understand, which makes it beginner-friendly.
2. **Interpreted Language:** Python code is executed line by line, making debugging easier.
3. **Dynamically Typed:** You don't need to declare variable types; Python determines the type at runtime.
4. **Cross-Platform:** Python can run on different operating systems, such as Windows, macOS, and Linux.
5. **Extensive Standard Library:** Python comes with a vast set of built-in libraries for tasks like file I/O, regular expressions, and networking.
6. **Large Ecosystem:** Python has third-party libraries like NumPy, Pandas, TensorFlow, and Django for web development, data analysis, machine learning, and more.
7. **Object-Oriented and Functional Programming:** Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

Common Uses of Python:

1. **Web Development:** Frameworks like Django and Flask make web development fast and scalable.
2. **Data Science and Machine Learning:** Libraries like NumPy, Pandas, Scikit-learn, and TensorFlow make Python a go-to language for data analysis, machine learning, and AI.

3. Automation/Scripting: Python is often used for automating repetitive tasks, from file handling to web scraping.
4. Game Development: Libraries like Pygame allow game developers to create simple games.
5. Desktop GUI Development: Tkinter, PyQt, and Kivy enable building cross-platform desktop applications.
6. Embedded Systems: MicroPython and CircuitPython are used for programming on microcontrollers.

PROJECT OVERVIEW

The main feature are :

- **WELCOME PAGE**
- **ADMIN PAGE**
- **MEMBERSHIP PAGE**

WELCOME PAGE

The aim of this code is to create a welcome screen for a Library Management System using Tkinter. It displays a window with a background image and a "Welcome" message, along with a button that allows the user to proceed to the login page by running another Python script ('loginpage.py').

The main goal of this code is to provide a simple graphical interface for the Library Management System's opening screen, allowing users to start the application and proceed to the login page by clicking the "OPEN" button. This serves as an entry point into the system, creating a user-friendly experience.

1. Tkinter Window Creation ('root'):

- Purpose: The `Tk()` function initializes the main window, setting its title to "Library Management" and its dimensions to 800x600 pixels.
- `root.resizable(False, False)`: Ensures the window size cannot be resized.

2. 'Background Image':

- Purpose: The background image (`librarybg.png`) is loaded using `PhotoImage` and applied to a `Label`. This label covers the entire window by using `place(relwidth=1, relheight=1)`.

3. Label ("Welcome to the Library Management System"):

- Purpose: Displays a welcome message to the user in the center of the window, with custom font, background, and foreground colors.

- `'label.place(relx=0.5, rely=0.4, anchor="center")'`: Positions the label in the middle of the window horizontally (`relx=0.5`) and slightly above the middle vertically (`rely=0.4`), using the `'anchor="center"'` option to center the text.

4. Login Button:

- Purpose: The button labeled "OPEN" allows the user to move from the welcome screen to the login page.

- `'command=on_log'`: When clicked, the button calls the `'on_log()'` function, which transitions the user to the login page.

- The button is centered horizontally (`relx=0.5`) and placed below the welcome message (`rely=0.5`).

5. `'on_log()'` Function:

- *Purpose: Handles the transition from the current Tkinter window to the login page.

- `'root.destroy()'`: Closes the current window.

- `'sp.run(["python", "loginpage.py"])'`: Launches the login page by running the `'loginpage.py'` script in a new process using the `'subprocess'` module.

ADMIN PAGE

The aim of this code is to provide a **Library Management System** GUI where an admin can **view**, **search**, **add**, **remove**, and **modify books** using a Tkinter-based graphical interface. It also integrates with a MySQL database to manage the book records stored in the `library_management` database.

The main aim of admin page is to create an **admin interface for managing a library system**. The admin can:

- **View** all books in the library,
- **Search** for specific books by title,

- **Add** new books,
- **Remove** books by their ID,
- **Modify** existing book details.

All these operations are backed by a **MySQL database** where the book records are stored. The system provides an easy-to-use graphical interface for these administrative tasks.

Key Aims and Functionality:

Main Window Setup:

Title and Layout: The main window has a background image and a title "CHETTINAD LIBRARY." There are buttons that allow the admin to perform different operations related to books.

Buttons for Admin Operations: The main window contains buttons to trigger various book-related operations like view, search, add, remove, and modify.

Database Integration:

The code uses `mysql.connector` to connect to a MySQL database called `library_management`.

All the operations (viewing, searching, adding, deleting, and modifying books) are performed directly on the books table in the database.

Functionality of the System :

Search Books (search_window): Opens a new window where the admin can search for books by title. The results are fetched from the database and displayed in a Treeview widget

View Books (view_window): Opens a window that shows all the books available in the database, with details like ID, Title, Cost, Availability, Author, and Genre.

Add Book (add_window): Provides an interface for the admin to input the details of a new book. The addvaluestodb() function inserts the data into the books table in the database.

Remove Book (remove_window): Opens a window where the admin can input a book ID to remove the book from the database. The removebookfromdb() function deletes the book record from the books table.

Modify Book (modify_window): Allows the admin to modify the details of an existing book by providing the book ID and the new details. The modifydb() function updates the record in the database.

Login Transition:

The openlogin() function closes the current window and runs another Python script (loginpage.py), which likely returns the user to the login screen.

MEMBERSHIP PAGE

The aim of this code is to create a Library Management System for members using Tkinter for the graphical user interface (GUI) and MySQL for database management.

The main goal of the code is to provide library members with a GUI-based interface where they can:

- Browse and search books.
- Borrow and return books with restrictions (one book per member at a time).
- View their current borrow history.

This code is part of a member-specific system, designed to streamline the process of borrowing and returning books in a library setting.

The system allows library members to:

1. View All Books: The system retrieves and displays all available books in the library, showing information such as book ID, title, cost, availability, author, and genre.

2. Search Books: Members can search for a specific book by its title, and the system will display the search results if the book is found in the database.

3. Request a Book: Members can request to borrow a book if it is available and they haven't already borrowed one. The system limits users to borrowing only one book at a time.

4. View Leased Books: Members can view the books they have currently borrowed, along with the request date and return date (if applicable).

5. Return a Book: Once the member is done with a book, they can return it, which marks the book as available in the library and removes the record from the borrow table.

Key Features:

1. Database Integration:

- The code uses MySQL to store and manage the data for books and borrow requests.
- There are two primary tables:
 - 'books': Stores information about all books in the library (ID, title, cost, availability, author, genre).
 - 'barter_books': Stores information about books currently borrowed by members, including the username, book ID, and request/return dates.

2. Book Operations:

- View All Books: Displays all books in a scrollable list using `Treeview` in Tkinter.
- Search Books: Searches for a book by title and displays the result if it exists.
- Request Book: Members can borrow a book if available. The system prevents borrowing more than one book at a time.
- View Leased Books: Displays books currently borrowed by the member.

- Return Book: Allows members to return a borrowed book, updating the database accordingly.

3. Graphical User Interface (GUI):

- The system uses Tkinter to create a simple, user-friendly interface for members.
- The main window features buttons that trigger different functions, such as viewing books, searching for books, requesting books, and returning them.

4. Member-Based System:

- The system assumes a single logged-in member ('username' is set to "SampleUser" as a placeholder). This would typically be replaced by a proper login mecha

FUNCTIONS USED

- The functions used for *welcome page* are :

1. **on_log()** Function

- Purpose: This function is responsible for handling the transition to the login page when the "OPEN" button is clicked.
- Functionality:
 - **root.destroy()**: Closes the current main window ('root') of the application.
 - **sp.run(['python', 'loginpage.py'])**: Runs the 'loginpage.py' Python script using the `subprocess` module. This is assumed to launch the login page for the Library Management System.

2. **root = Tk()**

- Purpose: Initializes the main window of the Tkinter application.
- Functionality:
 - **root.title("Library Management")**: Sets the title of the window to "Library Management".
 - **root.geometry("800x600")**: Defines the size of the window (800 pixels wide by 600 pixels tall).
 - **root.resizable(False, False)**: Prevents the window from being resized, keeping it fixed at the specified dimensions.

3. **Background Image Setup**

- Purpose: To load and display a background image in the main window.
- Components:
 - **background1 = PhotoImage(file="librarybg.png")**: Loads an image file ('librarybg.png') to use as the background.
 - **backgroundlabel = Label(root, image=background1)**: Creates a Label widget that contains the background image.

- **backgroundlabel.place(relwidth=1, relheight=1)**: Places the background image to cover the entire window (relative width and height set to 1 to fill the window).

4. 'label' (Welcome Message)

- Purpose: Displays a welcome message to the user in the center of the screen.
- Components:
 - **label = Label(root, text="Welcome to the Library Management System", font=("New Amsterdam", 20, "bold"), bg="black", fg="white")**: Creates a label with the welcome text, using custom fonts and colors (black background, white text).
 - **label.place(relx=0.5, rely=0.4, anchor="center")**: Places the label at the center horizontally (`relx=0.5`) and slightly above the center vertically (`rely=0.4`) of the window.

5. 'login_button' (Button to Open Login Page)

- Purpose: Creates a button labeled "OPEN" that, when clicked, triggers the 'on_log()' function to open the login page.
- Components:
 - **login_button = Button(root, text="OPEN", bg="black", fg="white", command=on_log, font=("New Amsterdam", 12, "bold"))**: Creates a button with the text "OPEN", black background, white text, and bold font. The button's 'command' is set to the 'on_log()' function, meaning it will execute that function when clicked.
 - **login_button.place(relx=0.5, rely=0.5, anchor="center")**: Places the button at the center of the screen horizontally and vertically.

6. root.mainloop()

- Purpose: Starts the Tkinter event loop, which waits for user interactions (like button clicks) and keeps the window open until it is closed by the user or through a function like 'root.destroy()'.

- The functions used in *admin page* are :

Here is the purpose of each function in the provided code:

1. **openlogin() Function** -

Purpose: Transitions the user to the login page.

- Functionality:
 - **root.after(3000, root.destroy()):** Closes the current window after 3 seconds.
 - **subprocess.run(["python", "loginpage.py"]):** Opens and runs the 'loginpage.py' script, transitioning the user back to the login page.

2. **search_window() Function**

- Purpose: Opens a new window where users can search for books by title.
- Functionality:
 - Creates a new 'Toplevel' window named "Search Books."
 - Provides a search input field and button.
 - Fetches search results from the MySQL database using the book's title and displays the result in a 'Treeview' widget.

- **Nested 'addBooks()' Function:**
 - Purpose: Handles searching for the book.
 - Functionality:
 - Queries the database for the searched book name.
 - Displays the search result in the 'Treeview' widget or shows an error message if no book is found.

3. **view_window() Function**

- Purpose: Opens a window to display all books available in the database.
- Functionality:
 - Creates a 'Toplevel' window titled "View Books."
 - Fetches and displays all book records from the 'books' table in a 'Treeview' widget.

- Adds a scrollbar to allow viewing all the records if the list exceeds the window's height.

4. add_window() Function

- Purpose: Opens a window where users can input details to add a new book to the database.
- Functionality:
 - Creates a 'Toplevel' window titled "Add Book."
 - Provides input fields for entering book details (ID, Title, Cost, Author, Genre, and Availability).
 - Adds a button that triggers the 'addvaluestodb()' function to insert the new book into the database.

5. addvaluestodb(bookid, title, cost, availability, author, genre) Function

- Purpose: Adds the provided book details to the database.
- Functionality:
 - Executes an SQL 'INSERT' query to add the book information into the 'books' table.
 - Commits the transaction if successful or shows an error message in case of failure.
 - Displays a success message after successfully adding the book to the database.

6. remove_window() Function

- Purpose: Opens a window where users can input a book ID to remove a book from the database.
- Functionality:
 - Creates a 'Toplevel' window titled "Remove Book."
 - Provides an input field for the user to enter the book ID.
 - Adds a button to trigger the 'removebookfromdb()' function to delete the book from the database.

7. removebookfromdb(book_id) Function

- Purpose: Removes the specified book from the database.

- Functionality:
 - Executes an SQL 'DELETE' query to remove the book based on the provided 'book_id'.
 - Asks for user confirmation before deleting the book.
 - Displays a success message if the book is removed successfully or an error message if it fails.

8. **modify_window() Function**

- Purpose: Opens a window where users can modify details of an existing book.
- Functionality:
 - Creates a 'Toplevel' window titled "Modify Book."
 - Provides input fields for the user to update the book's details (ID, Title, Cost, Availability, Author, and Genre).
 - Adds a button that triggers the 'modifydb()' function to update the book's information in the database.

9. **modifydb(book_id, title, cost, available, author, genre) Function**

- Purpose: Updates the specified book's details in the database.
- Functionality:
 - Executes an SQL 'UPDATE' query to modify the details of the book in the 'books' table.
 - Commits the transaction if successful or shows an error message if the operation fails.
 - Displays a success message after successfully modifying the book.

10. Main Window Components:

- **Tkinter Window ('root'):**
 - Purpose: Creates the main window for the Library Management System application.
 - Functionality: Sets up the main window with a background image, a title ("CHETTINAD LIBRARY"), and a button frame.
- **Background Image ('background_image'):**

- Purpose: Displays a background image in the main window.
- Functionality: Loads and displays the 'library_background_1920x1024.png' image in the main window using a 'Label'.

- Title Label ('title_label'):

- Purpose: Displays the title "CHETTINAD LIBRARY" at the top of the main window.

- Functionality: Uses the 'Label' widget to display the title text with specified font and color.

- Button Frame ('button_frame'):

- Purpose: Holds the action buttons (View, Search, Add, Remove, Modify).

- Functionality: Organizes the buttons vertically and provides padding for a structured layout.

- Action Buttons:

- **View Button:** Opens the 'view_window()' to display all books.

- **Search Books Button:** Opens the 'search_window()' to search for a specific book.

- **Add Book Button:** Opens the 'add_window()' to add a new book.

- **Remove Book Button:** Opens the 'remove_window()' to remove a book.

- **Modify Book Button:** Opens the 'modify_window()' to modify book details.

- **Back Button:** Runs the 'openlogin()' function to return to the login page.

- The functions used in *membership page* are :

1. view_window() Function

- Purpose: Opens a new window displaying all books available in the database.

- Functionality:

- Creates a new 'Toplevel' window titled "View Books."

- Displays all books from the 'books' table in a 'Treeview' widget with columns for ID, Title, Cost, Availability, Author, and Genre.

- Adds a scrollbar for easy navigation through the list of books.

2. search_window() Function

- Purpose: Allows the user to search for a specific book by its name.
- Functionality:
 - Opens a new 'Toplevel' window titled "Search Books."
 - Provides a search input field for entering the book's title and a search button to trigger the search.
 - Executes a query to fetch the book from the 'books' table based on the book name.
 - Displays the result in a 'Treeview' widget or shows an error message if the book is not found.

-Nested addBooks() Function:

- Purpose: Handles searching for the book and updating the search results in the 'Treeview' widget.
- Functionality: Queries the 'books' table, retrieves the search result, and updates the 'Treeview' to display the result or shows an error if no book is found.

3. request_book() Function:

- Purpose: Allows the user to request a book from the library.
- Functionality:
 - Opens a new 'Toplevel' window titled "Request Book."
 - Provides an input field for the user to enter the book ID they want to borrow.
 - Checks if the user has already borrowed a book (limits borrowing to one book at a time).
 - Verifies if the requested book is available, and if so, inserts the request into the 'barter_books' table and updates the 'books' table to mark the book as unavailable.

- Nested request_confirm() Function:

- Purpose: Confirms the book request and updates the database.
- Functionality:
 - Validates the book ID.

- Ensures the user has not already borrowed a book.
- Marks the book as unavailable in the 'books' table and inserts the request in the 'barter_books' table.

4. view_leased_books() Function:

- Purpose: Displays the list of books the user has currently borrowed.
- Functionality:
 - Opens a new 'Toplevel' window titled "Books Leased."
 - Fetches the user's borrowed books from the 'barter_books' table and displays them in a 'Treeview' widget with columns for Book ID, Book Name, Request Date, and Return Date.

5. return_book() Function:

- Purpose: Allows the user to return a borrowed book.
- Functionality:
 - Opens a new 'Toplevel' window titled "Return Book."
 - Provides an input field for the user to enter the book ID they wish to return.
 - Checks if the book is in the user's borrowed list in the 'barter_books' table.
 - If the book is found, it marks the book as available again in the 'books' table and deletes the entry from the 'barter_books' table.

- Nested return_confirm() Function:

- Purpose: Confirms the return of the book and updates the database.
- Functionality:
 - Verifies that the user has borrowed the specified book.
 - Marks the book as available in the 'books' table and removes the record from the 'barter_books' table.

6. Main Window Components

- Tkinter Window ('root'):

- Purpose: Creates the main window for the Library Management System's member interface.

-Functionality: Initializes the window size, sets the title, and ensures the window is not resizable.

- Background Image ('background1'):

- Purpose: Sets the background image for the main window.
- Functionality: Loads and displays the 'library_background_1920x1024.png' image in the main window using a 'Label'.

- Button Frame ('button_frame'):

- Purpose: Organizes the buttons for various user actions (viewing, searching, requesting, returning books).
- Functionality: Groups and places the buttons vertically with padding.

- Action Buttons:

- **View All Books Button:** Opens the 'view_window()' to display all books in the library.
- **Search Books Button:** Opens the 'search_window()' to search for specific books.
- **Request Book Button:** Opens the 'request_book()' window to request a book.
- **Books Leased Button:** Opens the 'view_leased_books()' window to view books currently borrowed by the user.
- **Return Book Button:** Opens the 'return_book()' window to return a borrowed book.

TABLES USED

❖ The tables used in *admin page* are:

Books table

Field	Type	Null	Key	Default	Extra
book_id	int	NO	PRI	NULL	
book_name	varchar(40)	YES		NULL	
cost	int	YES		NULL	
avaiable	tinyint(1)	NO		NULL	
author	char(40)	YES		NULL	
genre	char(20)	YES		NULL	

1. **book_id (int):**

- Purpose: A unique identifier for each book.
- Attributes:
 - Type: 'int' (integer).
 - Null: 'NO' (cannot be NULL, meaning each book must have an ID).
 - Key: 'PRI' (Primary Key, ensures that each 'book_id' is unique).

2. **book_name (varchar(40)):**

- Purpose: The name of the book.
- Attribute:
 - Type: 'varchar(40)' (a string with a maximum of 40 characters).
 - Null: 'YES' (can be NULL, meaning a book name is optional).

3. **cost (int):**

- Purpose: The cost of the book.
- Attributes:
 - Type: 'int' (integer).
 - Null: 'YES' (can be NULL, meaning the cost is optional).

4. **avaiable (tinyint(1)):**

- Purpose: Indicates whether the book is available or not.

- **Attributes:**

- Type: 'tinyint(1)' (a small integer, typically used for boolean values, where 1 means available and 0 means not available).

- Null: 'NO' (cannot be NULL, meaning availability information must be provided for each book).

5. author (char(40)):

- Purpose: The author of the book.

- Attributes:

- Type: 'char(40)' (a string with exactly 40 characters).

- Null: 'YES' (can be NULL, meaning the author information is optional).

6. genre (char(20)):

- Purpose: The genre of the book (e.g., Fiction, Non-fiction, Mystery).

- Attributes:

- Type: char(20) (a string with exactly 20 characters).

- Null: YES (can be NULL, meaning genre information is optional).

General Purpose of the Table:

This table is designed to store information about books in a library management system. It contains fields for:

- Unique identification ('book_id').

- Book-specific information like the name, cost, and availability status.

- Author and genre details, which help classify and search for books.

❖ The table used in *membership page* are:

Members table

Field	Type	Null	Key	Default	Extra
UserName	varchar(20)	YES		NULL	
Password	varchar(10)	YES		NULL	

Table Purpose:

This table is used to store login credentials for users in a system, where each user has a username and a password.

1.UserName (varchar(20)):

- Purpose: Stores the username for the user.
- Attributes:
 - Type: 'varchar(20)' (a variable-length string with a maximum of 20 characters).
 - Null: 'YES' (can be 'NULL', meaning a username is optional).
 - Default: 'NULL', meaning if no username is provided, it will default to 'NULL'.

2. Password (varchar(10)):

- Purpose: Stores the password for the user.
- Attributes:
 - Type: 'varchar(10)' (a variable-length string with a maximum of 10 characters).
 - Null: 'YES' (can be 'NULL', meaning a password is optional).
 - Default: 'NULL', meaning if no password is provided, it will default to 'NULL'.

General Purpose of the Table:

- This table is designed to store user credentials for an application or a system.
- It keeps track of usernames and passwords, which are essential for logging in to the system.
- Since both fields allow 'NULL' values, it might be used in situations where not all users are required to have login credentials immediately, or additional validation logic might be needed in the application layer to prevent incomplete records.

Barter table

Field	Type	Null	Key	Default	Extra
Username	varchar(30)	NO	PRI	NULL	
book_id	int	YES		NULL	
book_name	varchar(40)	YES		NULL	
req_date	date	YES		NULL	
return_date	date	YES		NULL	

Table Purpose:

This table is used to store records of books borrowed by users. It keeps track of which user has borrowed which book, along with the dates related to borrowing and returning the book.

1. Username (varchar(30)):

- Purpose: Stores the username of the person who borrowed the book.
- Attributes:
 - Type: 'varchar(30)' (a string with a maximum of 30 characters).
 - Null: 'NO' (cannot be NULL, meaning each record must be associated with a username).
 - Key: 'PRI' (Primary Key, likely used in combination with 'book_id' to uniquely identify a borrowing record).

2. **book_id (int):**

- Purpose: Stores the unique identifier (ID) of the borrowed book.
- Attributes:
 - Type: 'int' (integer).
 - Null: 'YES' (can be NULL, meaning this value is optional).
 - This field identifies the book being borrowed by the user.

3. **book_name (varchar(40)):**

- Purpose: Stores the name of the borrowed book.
- Attributes:
 - Type: 'varchar(40)' (a string with a maximum of 40 characters).
 - Null: 'YES' (can be NULL, meaning the book name is optional).
 - This field provides an easy-to-read description of the borrowed book.

4. **req_date (date):**

- Purpose: Stores the date when the book was borrowed (request date).
- Attributes:
 - Type: 'date' (a field for storing date values).
 - Null: 'YES' (can be NULL, meaning the request date is optional).

5. **return_date (date):**

- Purpose: Stores the date when the book was returned.
- Attributes:
 - Type: 'date' (a field for storing date values).
 - Null: 'YES' (can be NULL, meaning the return date is optional).
 - This field will be populated once the book is returned.

General Purpose of the Table:

This table is designed to track which users have borrowed which books, along with the dates the books were borrowed (request date) and returned (return date). It could be used to:

- Identify overdue books by comparing the current date with the 'req_date' or 'return_date'.
- Limit the number of books a user can borrow at a time.
- Keep a historical record of all borrowings for reporting purposes.

By using the 'Username' and 'book_id' together as a primary key ('PRI'), the table ensures that the same user cannot borrow the same book more than once at the same time.

SOURCE CODE

★ Source code for *welcome page* :

```
from tkinter import *
import subprocess as sp

# Function to run the login page
def on_log():
    root.destroy()
    sp.run(["python", "loginpage.py"])

root = Tk()
root.title("Library Management")
root.geometry("800x600")
root.resizable(False, False)

# Set background image
background1 = PhotoImage(file="librarybg.png")
backgroundlabel = Label(root, image=background1)

backgroundlabel.place(relwidth=1, relheight=1)

# Create a label that will "blend" with the background image
#background2=PhotoImage(file="library2.png")

label = Label(root, text="Welcome to the Library Management System",
font=("New Amsterdam", 20, "bold"),bg="black",fg="white")
label.place(relx=0.5, rely=0.4, anchor="center")
```

```
# Create login button
login_button = Button(root, text="OPEN",bg="black",fg="white",
command=on_log,font=("New Amsterdam", 12, "bold"))
login_button.place(relx=0.5, rely=0.5, anchor="center")

root.mainloop()
```

★ Source code for *admin page* :

```
from tkinter import *
from tkinter import ttk
import mysql.connector as sql
import tkinter.messagebox as messagebox
import subprocess

mycon=sql.connect(host='localhost',username='root',password='root',database='library_management')
mycur=mycon.cursor()
#functions to search ,add ,delete,modify

def openlogin():
    root.after(3000,root.destroy())
    subprocess.run(["python","loginpage.py"])

def search_window():
    search = Toplevel(root)
    search.title("Search Books")
    search.geometry("800x400")
    search.configure(bg="lightblue")

    Label(search,text="Search Books").pack(pady=5)
    search_entry=Entry(search)
```



```
search_entry.pack(pady=5)
```

```
def addBooks():
```

```
    #mysql
```

```
    searchdt=search_entry.get().lower()
```

```
    m="SELECT * FROM books WHERE book_name=%s"
```

```
    mycur.execute(m,(searchdt,))
```

```
    result=mycur.fetchall()
```

```
    for item in tree.get_children():
```

```
        tree.delete(item)
```

```
    if result:
```

```
        for row in result:
```

```
            tree.insert("", "end", values=row)
```

```
    else:
```

```
        messagebox.showerror("ERROR", "No book found")
```

```
columns = ("ID", "Title", "Cost", "Availability", "Author", "Genre")
```

```
tree = ttk.Treeview(search, columns=columns, show='headings')
```

```
#search button
```

```
Button(search,text="Search",anchor="center",relief="groove",command=addBooks).pack(pady=5)
```

```
for col in columns:
```

```
    tree.heading(col,text=col)
```

```
    tree.column(col,anchor=CENTER,width=100)
```

```
tree.pack(fill=BOTH,expand=True)
```

```
def view_window():
```

```
    view =Toplevel(root)
```

```

view.title("View Books")
view.geometry("800x400")
view.configure(bg="lightblue")

columns = ("ID", "Title", "Cost", "Availability", "Author", "Genre",)
tree = ttk.Treeview(view, columns=columns, show='headings')

for col in columns:
    tree.heading(col, text=col)
    tree.column(col, anchor='center', width=100)

scrollbar = Scrollbar(view, orient="vertical", command=tree.yview)
tree.configure(yscroll=scrollbar.set)
scrollbar.pack(side=RIGHT, fill=Y)

ask="select * from books"
mycur.execute(ask)
result=mycur.fetchall()

for item in result:
    tree.insert("", 'end', values=item)

tree.pack(fill=BOTH, expand=True)

def add_window():
    addw = Toplevel(root)
    addw.title("Add Book")
    addw.geometry("600x500")
    addw.configure(bg="lightgreen")

    Label(addw, text="BookID:", bg="lightgreen").pack(pady=5)

```

```
bookid_entry = Entry(addw)
bookid_entry.pack(pady=5)
```

```
Label(addw, text="Title:", bg="lightgreen").pack(pady=5)
title_entry = Entry(addw)
title_entry.pack(pady=5)
```

```
Label(addw, text="Cost", bg="lightgreen").pack(pady=5)
Cost_entry = Entry(addw)
Cost_entry.pack(pady=5)
```

```
Label(addw, text="Author:", bg="lightgreen").pack(pady=5)
author_entry = Entry(addw)
author_entry.pack(pady=5)
```

```
Label(addw, text="Genre:", bg="lightgreen").pack(pady=5)
genre_entry = Entry(addw)
genre_entry.pack(pady=5)
```

```
Label(addw, text="Availability:", bg="lightgreen").pack(pady=5)
availability_entry = Entry(addw)
availability_entry.pack(pady=5)
```

```
Button(addw, text="Add Book", command=lambda:
addvaluestodb(bookid_entry.get(),title_entry.get(),Cost_entry.get(),availability_entr
y.get(), author_entry.get(), genre_entry.get()), bg="white").pack(pady=20)
```

```
def addvaluestodb(bookid, title, cost, availability, author, genre):
    try:
        st = "INSERT IGNORE INTO books
(book_id,book_name,cost,avaiable,author,genre) VALUES (%s, %s, %s, %s,
%s, %s)"
        mycur.execute(st, (bookid, title, cost, availability, author, genre))
```

```
mycon.commit()
messagebox.showinfo("Success"," Book is added successfully ")
except Exception as e:
    mycon.rollback()
messagebox.showerror("ERROR","PLEASE ENTER DATA PROPERLY")
```

#function to open the remove window

```
def remove_window():
```

```
    removew = Toplevel(root)
    removew.title("Remove Book")
    removew.geometry("400x200")
    removew.configure(bg="lightcoral")
```

```
Label(removew, text="Enter Book ID to Remove:",
bg="lightcoral").pack(pady=20)
book_id_entry = Entry(removew)
book_id_entry.pack(pady=10)
```

```
Button(removew,text="Remove",command=lambda:removebookfromdb(book_id_entry.get()), bg="white").pack(pady=20)
```

```
def removebookfromdb(book_id):
```

```
    c=messagebox.askquestion("Remove"," Are you Sure?")
```

```
    try:
```

```
        if c=="yes":
```

```
            remove="delete from books where book_id=%s"
```

```
            mycur.execute(remove,(book_id,))
```

```
            mycon.commit()
```

```
            messagebox.showinfo("Success!!"," Book is removed ")
```

```
    except Exception as e:
```

```
        mycon.rollback()
```

```
messagebox.showerror("ERROR","Book ID not available")
```

```
def modify_window():
```

```
    modify_window = Toplevel(root)
```

```
    modify_window.title("Modify Book")
```

```
    modify_window.geometry("600x500")
```

```
    modify_window.configure(bg="lightyellow")
```

```
    #ui desgin
```

```
    Label(modify_window,text="BookID",bg="lightyellow").pack(pady=5)
```

```
    bookid_entry=Entry(modify_window)
```

```
    bookid_entry.pack(pady=5)
```

```
    Label(modify_window,text="Title",bg="lightyellow").pack(pady=5)
```

```
    Title_entry=Entry(modify_window)
```

```
    Title_entry.pack(pady=5)
```

```
    Label(modify_window,text="Cost",bg="lightyellow").pack(pady=5)
```

```
    cost_entry=Entry(modify_window)
```

```
    cost_entry.pack(pady=5)
```

```
    Label(modify_window,text="Available",bg="lightyellow").pack(pady=5)
```

```
    available_entry=Entry(modify_window)
```

```
    available_entry.pack(pady=5)
```

```
    Label(modify_window,text="Author",bg="lightyellow").pack(pady=5)
```

```
    author_entry=Entry(modify_window)
```

```
    author_entry.pack(pady=5)
```

```
    Label(modify_window,text="Genre",bg="lightyellow").pack(pady=5)
```

```
    genere_entry=Entry(modify_window)
```

```
    genere_entry.pack(pady=5)
```

```

modifybutton=Button(modify_window,text="Modify",relief="raised",anchor
=CENTER,command=lambda:modifydb(bookid_entry.get(),Title_entry.get()
,cost_entry.get(),avaiable_entry.get(),author_entry.get(),genare_entry.get()))
modifybutton.pack(pady=5)

```

```

def modifydb(book_id,title,cost,available,author,genre):

```

```

    try:
        s1="update books set
book_name=%s,cost=%s,avaiaible=%s,author=%s,genre=%s where book_id=%s"
        mycur.execute(s1,(title,cost,available,author,genre,book_id))
        mycon.commit()
        messagebox.showinfo("SUCCESS","Modified successfully")
    except Exception as e:
        mycon.rollback()
        messagebox.showerror("ERROR","Please Make Sure to Enter All
data")
#main
root = Tk()
root.title("Library Management System")
root.geometry("800x600")

#background work
background_image
=PhotoImage(file="library_background_1920x1024.png")
background_label =Label(root, image=background_image)
background_label.place(relwidth=1, relheight=1)

```

```

#title
title_label = Label(root, text="CHETTINAD LIBRARY", font=("Arial", 24,
"bold"), bg="#4CAF50", fg="white")
title_label.pack(pady=20)

#buttons for admin
button_frame = Frame(root,bg="brown")
button_frame.pack(pady=70)

#btn_style = {"font": ("Arial", 16), "width": 20, "bg": "#4CAF50", "fg":
"white"}
Button(button_frame,          text="View",          command=view_window,
font=("Arial",16),fg="white",bg="black",relief="groove",width=20).pack(pa
dy=10)
Button(button_frame,  text="Search  Books",  command=search_window,
font=("Arial",16),fg="white",bg="black",relief="groove",width=20).pack(pa
dy=10)
Button(button_frame,          text="          Add          Book          ",
command=add_window,font=("Arial",16),fg="white",bg="black",relief="gr
oove",width=20).pack(pady=10)
Button(button_frame,  text="Remove  Book",  command=remove_window,
font=("Arial",16),fg="white",bg="black",relief="groove",width=20).pack(pa
dy=10)
Button(button_frame,  text="Modify  Book",  command=modify_window,
font=("Arial",16),fg="white",bg="black",relief="groove",width=20).pack(pa
dy=10)
Button(button_frame,  text="Back",bg="black",fg="white",relief="sunken",
font=("Arial", 12, "bold"),command=lambda:openlogin()).pack(pady=10)

root.mainloop()

```

★ The source for *membership page* :

from tkinter import *

```

from tkinter import ttk
import tkinter.messagebox as messagebox
import mysql.connector as mc
from datetime import datetime

# Database connection
con = mc.connect(host='localhost', user='root', password='root',
charset='utf8')
cur = con.cursor()

# Use the library_management database
cur.execute("USE library_management")

# Create necessary tables if not present
cur.execute("""
CREATE TABLE IF NOT EXISTS barter_books(
    Username VARCHAR(30),
    book_id INT,
    book_name VARCHAR(40),
    req_date DATE,
    return_date DATE,
    PRIMARY KEY (Username, book_id))""")

cur.execute("""
CREATE TABLE IF NOT EXISTS books(
    book_id INT PRIMARY KEY,
    book_name VARCHAR(40),
    cost INT,
    available TINYINT(1),
    author CHAR(40),
    genre CHAR(20))""")

```



```
# Placeholder for logged-in username (for demonstration purposes)
username = "SampleUser" # Replace with actual login mechanism

# Function to view all available books
def view_window():
    view = Toplevel(root)
    view.title("View Books")
    view.geometry("800x400")
    view.configure(bg="lightblue")

    # Columns for the Treeview
    columns = ("ID", "Title", "Cost", "Availability", "Author", "Genre")
    tree = ttk.Treeview(view, columns=columns, show='headings')

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, anchor='center', width=100)

    # Add scrollbar for the Treeview
    scrollbar = Scrollbar(view, orient="vertical", command=tree.yview)
    tree.configure(yscroll=scrollbar.set)
    scrollbar.pack(side=RIGHT, fill=Y)

    # Fetch all books from the database
    ask = "SELECT * FROM books"
    cur.execute(ask)
    result = cur.fetchall()

    for item in result:
        tree.insert("", 'end', values=item)

    tree.pack(fill=BOTH, expand=True)
```

```

# Function to search for books
def search_window():
    search = Toplevel(root)
    search.title("Search Books")
    search.geometry("800x400")
    search.configure(bg="lightblue")

    Label(search, text="Search Books").pack(pady=5)
    search_entry = Entry(search)
    search_entry.pack(pady=5)

    def addBooks():
        # Search for the book in the database
        search_term = search_entry.get().lower()
        query = "SELECT * FROM books WHERE
LOWER(book_name) = %s"
        cur.execute(query, (search_term,))
        result = cur.fetchone()

        # Ensure all results are processed
        con.commit()

        for item in tree.get_children():
            tree.delete(item)

        if result:
            tree.insert("", "end", values=result)
        else:
            messagebox.showerror("ERROR", "No book found")

    columns = ("ID", "Title", "Cost", "Availability", "Author", "Genre")
    tree = ttk.Treeview(search, columns=columns, show='headings')

```

```

# Search button
    Button(search, text="Search", anchor="center", relief="groove",
command=addBooks).pack(pady=5)

for col in columns:
    tree.heading(col, text=col)
    tree.column(col, anchor=CENTER, width=100)

tree.pack(fill=BOTH, expand=True)

# Function to request a book (only 1 book per user)
def request_book():
    req = Toplevel(root)
    req.title("Request Book")
    req.geometry("600x350")

    Label(req, text="REQUEST BOOK", bg="lightblue",
font=("Arial", 17)).grid(row=0, column=1, columnspan=2, pady=10,
padx=150)

    Label(req, text="Book Id", bg="lightblue", font=("Arial",
15)).grid(row=1, column=0, columnspan=2, pady=10)
    book_id_entry = Entry(req)
    book_id_entry.grid(row=1, column=2, padx=5, pady=5,
sticky="W")

def request_confirm():
    # Ensure book_id is an integer and valid
    try:
        book_id = int(book_id_entry.get())
    except ValueError:
        messagebox.showerror("ERROR", "Invalid Book ID. Please
enter a valid number.")

```

```

        return

        # Check if the user already has a book borrowed (where
        return_date is still NULL)
        cur.execute("SELECT * FROM barter_books WHERE Username
        = %s AND return_date IS NULL", (username,))
        existing_borrow = cur.fetchone()

        # Ensure all results are processed
        con.commit()

        if existing_borrow:
            messagebox.showerror("ERROR", "You have already
            borrowed a book. Please return it before borrowing another.")
            return

        # Check if the book is available
        cur.execute("SELECT * FROM books WHERE book_id = %s
        AND available = 1", (book_id,))
        book_data = cur.fetchone()

        # Ensure all results are processed
        con.commit()

        if not book_data:
            messagebox.showerror("ERROR", "Book not found or
            unavailable.")
            return

        # If everything is fine, proceed to borrow the book
        book_name = book_data[1]
        req_date = datetime.now().date()

```

```

# Insert the request into barter_books if no duplicate exists
cur.execute("""
            INSERT INTO barter_books (Username, book_id,
book_name, req_date, return_date)
            VALUES (%s, %s, %s, %s, NULL)
            """, (username, book_id, book_name, req_date))

cur.execute("UPDATE books SET available = 0 WHERE book_id
= %s", (book_id,))
con.commit()

messagebox.showinfo("Success", "Book requested successfully")
req.destroy()

Button(req, text="Request Book", font=("Arial", 12),
command=request_confirm).grid(row=3, column=2)

# Function to view leased books (only for the current logged-in
member)
def view_leased_books():
    leased_window = Toplevel(root)
    leased_window.title("Books Leased")
    leased_window.geometry("600x350")

    Label(leased_window, text="BOOKS LEASED", bg="lightblue",
font=("Arial", 17)).pack(pady=10)

    tree = ttk.Treeview(leased_window, columns=("Book ID", "Book
Name", "Request Date", "Return Date"), show='headings')
    tree.heading("Book ID", text="Book ID")
    tree.heading("Book Name", text="Book Name")
    tree.heading("Request Date", text="Request Date")
    tree.heading("Return Date", text="Return Date")

```

```

tree.pack(fill=BOTH, expand=True)

# Fetch books leased by the current user
cur.execute("SELECT book_id, book_name, req_date, return_date
FROM barter_books WHERE Username = %s", (username,))
rows = cur.fetchall()

# Ensure all results are processed
con.commit()

for row in rows:
    tree.insert("", "end", values=row)

# Function to return a book
def return_book():
    return_win = Toplevel(root)
    return_win.title("Return Book")
    return_win.geometry("600x350")

    Label(return_win, text="RETURN BOOK", bg="lightblue",
font=("Arial", 17)).grid(row=0, column=1, columnspan=2, pady=10,
padx=150)

    Label(return_win, text="Book Id", bg="lightblue", font=("Arial",
15)).grid(row=1, column=0, columnspan=2, pady=10)
    book_id_entry = Entry(return_win)
    book_id_entry.grid(row=1, column=2, padx=5, pady=5,
sticky="W")

def return_confirm():
    book_id = book_id_entry.get()

```

```

        # Check if the book exists in barter_books for the logged-in
member
        cur.execute("SELECT * FROM barter_books WHERE book_id
= %s AND Username = %s", (book_id, username))
        lease_data = cur.fetchone()

        # Ensure all results are processed
        con.commit()

        if not lease_data:
            messagebox.showerror("ERROR", "This book was not leased
by you")
            return

        # Mark the book as available in the books table
        cur.execute("UPDATE books SET avaiable = 1 WHERE book_id =
%s", (book_id,))

        # Delete the entry from barter_books since the book is returned
        cur.execute("DELETE FROM barter_books WHERE book_id =
%s AND Username = %s", (book_id, username))
        con.commit()

        messagebox.showinfo("Success", "Book returned and removed
from borrow records successfully")
        return_win.destroy()

        Button(return_win, text="Return Book", font=("Arial", 12),
command=return_confirm).grid(row=3, column=2)

# Main window
root = Tk()
root.title("Library Management - Member Page")

```

```
root.geometry("800x600")
root.resizable(False, False)
```

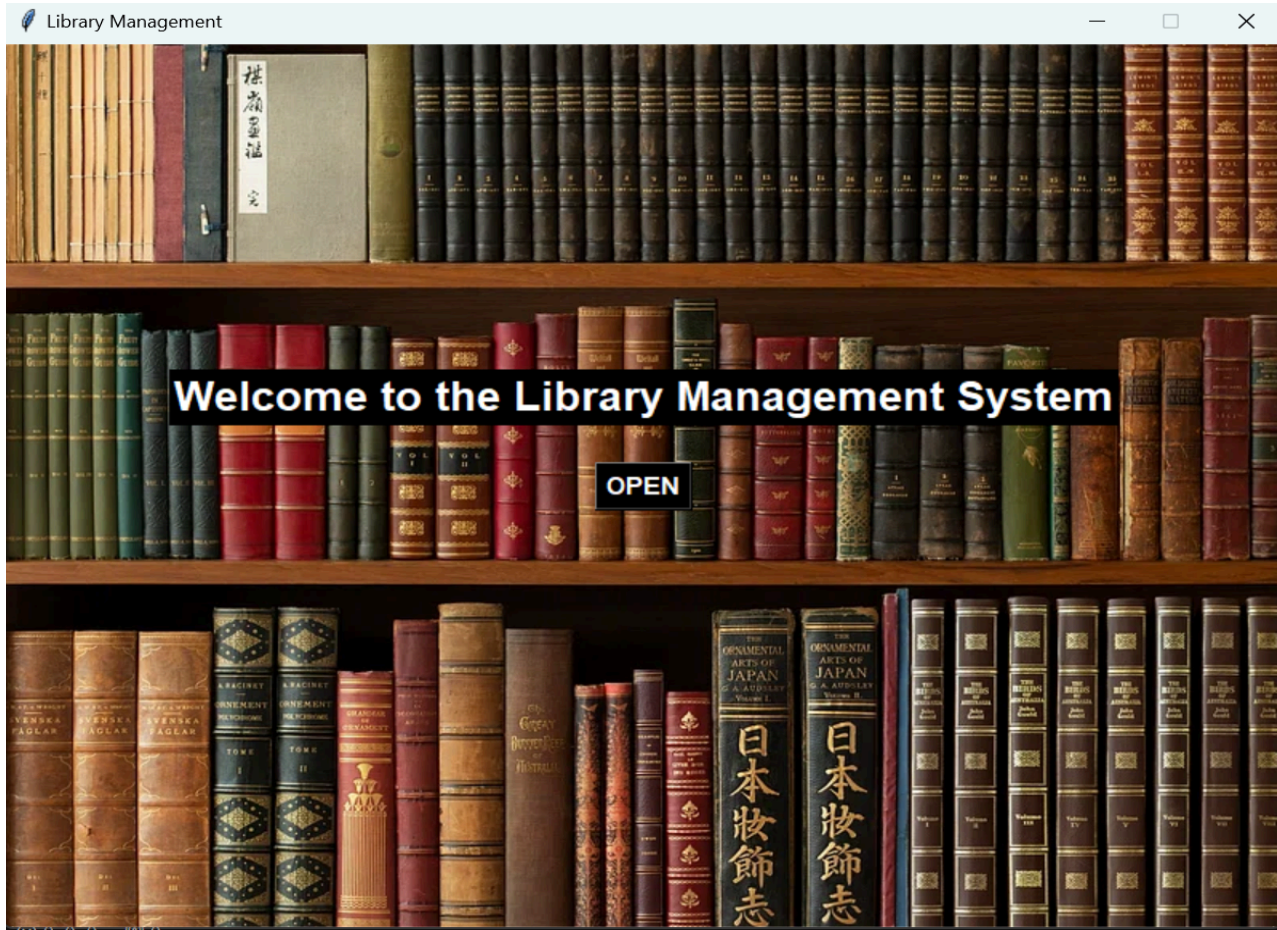
```
# Set background image (replace with the correct path to your image)
background1 =
PhotoImage(file="library_background_1920x1024.png")
backgroundlabel = Label(root, image=background1)
backgroundlabel.place(relwidth=1, relheight=1)
```

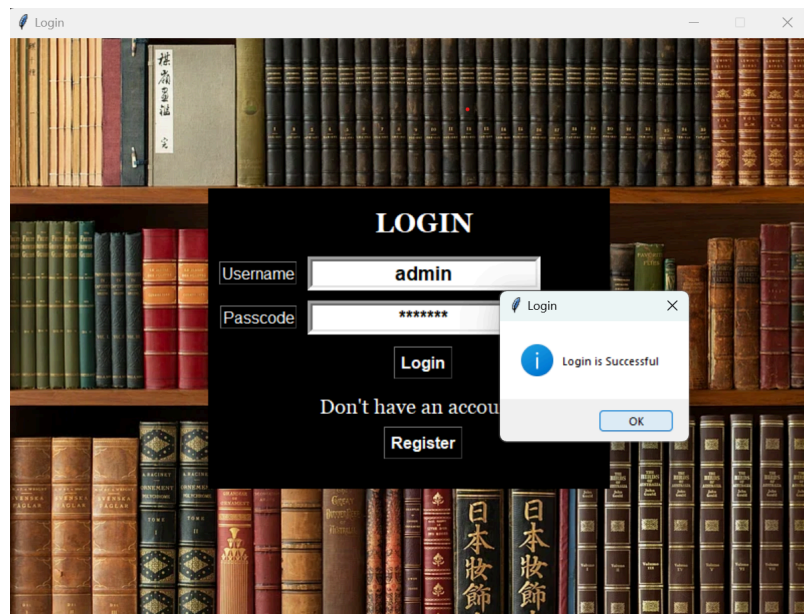
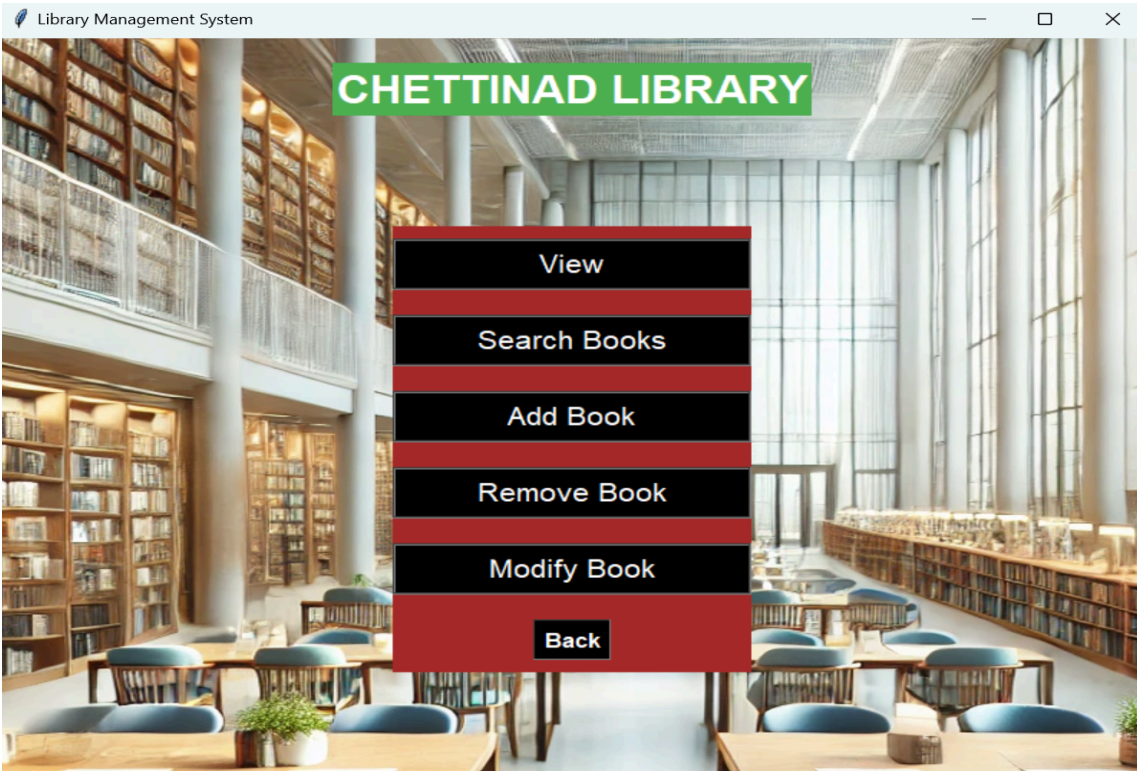
```
# Buttons for library actions
button_frame = Frame(root, bg="lightblue")
button_frame.pack(pady=100)
```

```
# Added the View All Books button above the Search Books button
Button(button_frame, text="View All Books",
command=view_window, font=("Arial", 16), fg="black", bg="white",
relief="groove", width=20).pack(pady=10)
Button(button_frame, text="Search Books",
command=search_window, font=("Arial", 16), fg="black",
bg="white", relief="groove", width=20).pack(pady=10)
Button(button_frame, text="Request Book", command=request_book,
font=("Arial", 16), fg="black", bg="white", relief="groove",
width=20).pack(pady=10)
Button(button_frame, text="Books Leased",
command=view_leased_books, font=("Arial", 16), fg="black",
bg="white", relief="groove", width=20).pack(pady=10)
Button(button_frame, text="Return Book", command=return_book,
font=("Arial", 16), fg="black", bg="white", relief="groove",
width=20).pack(pady=10)
```

```
Root.mainloop()
```


SAMPLE OUTPUT





View Books					
ID	Title	Cost	Availability	Author	Genre
102	1984	250	1	Ashwin	History
103	Pride and Prejudice	200	0	None	None
104	The Great Gatsby	270	1	None	None
105	Moby-Dick	320	1	None	None
106	War and Peace	450	0	None	None
107	The Catcher in the Rye	310	1	None	None
108	The Hobbit	280	1	None	None
109	Crime and Punishment	340	0	None	None
110	Jane Eyre	220	1	None	None
111	Brave New World	230	1	None	None
112	Wuthering Heights	210	0	None	None
113	The Odyssey	260	1	None	None
114	The Brothers Karamazov	400	1	None	None
115	Madame Bovary	290	0	None	None
116	The Iliad	310	1	None	None
117	Anna Karenina	330	1	None	None
118	The Divine Comedy	450	0	None	None
119	Don Quixote	350	1	None	None

Search Books					
Search Books					
1984					
Search					
ID	Title	Cost	Availability	Author	Genre
102	1984	250	1	Ashwin	History
133	1984	250	1	Ashwin	History

 Add Book

—

□

×

BookID:

Title:

Cost

Author:

Genre:

Availability:

Add Book

 Remove Book

—

□

×

Enter Book ID to Remove:

Remove

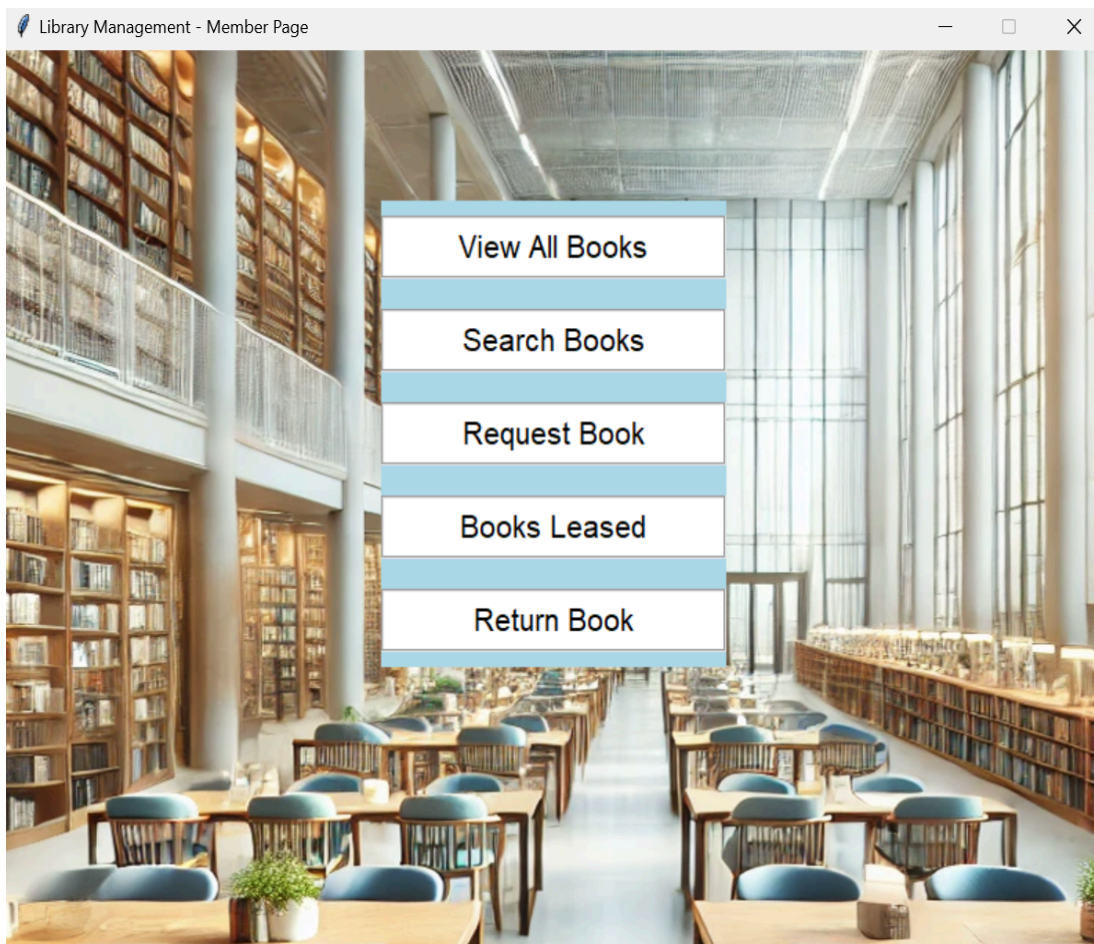
Register

Register

Username

Passcode

Confirm Passcode



Request Book

REQUEST BOOK

Book Id

Request Book

Success

 Book requested successfully

OK

Books Leased		
BOOKS LEASED		
Book ID	Book Name	Request Date
101	To Kill a Mockingbird	2024-10-13

Return Book

RETURN BOOK

Book Id

101

Return Book



Success



Book returned and removed from borrow records successfully

OK



Books Leased



BOOKS LEASED

Book ID	Book Name	Request Date
---------	-----------	--------------

CONCLUSION

This program is a **Library Management System** implemented using **Tkinter** for the graphical user interface (GUI) and **MySQL** for backend data management. The program allows both admins and members of a library to manage book-related tasks such as viewing, searching, borrowing, and returning books.

This program provides a basic yet functional library management system that can be used to handle book transactions in a library setting. It allows admins to manage the library's book inventory while enabling members to borrow and return books. The program is built with flexibility in mind, allowing for further customization and additional features.

Potential Extensions:

- **User Authentication:** Implementing a proper login system where admins and members have different credentials and access levels.
- **Overdue Notifications:** Adding a feature to notify members when they have overdue books.
- **Enhanced Search:** Expanding the search feature to allow users to search for books by author, genre, or availability.
- **Reports and Analytics:** Adding functionality to generate reports for admins, such as the most borrowed books or members with overdue books.

Workflow:

- **Login:** The system assumes a basic login mechanism with a placeholder for username (sample user), which can be replaced with an actual login process.
- **Admin vs Member Role:** The program could be easily extended to handle different user roles, where the admin has more permissions (like adding, removing, and modifying books) while members can only view, request, and return books.

- **Interaction with MySQL Database:** The program uses MySQL queries to interact with the database, ensuring that all book transactions are recorded and managed centrally in the library_management database.

BIBLIOGRAPHY

<https://docs.python.org/3/library/tkinter.html>

<https://dev.mysql.com/doc/refman/8.0/en/>

<https://github.com/>

<https://www.geeksforgeeks.org/python-gui-tkinter/>

